APL 405: Machine Learning

Lecture 5: Neural Network

By

Prof. Arpit Agrawal and Prof. Rajdip Nayek

Assistant Professor,
Department of Applied Mechanics,
IIT Delhi

Email: arpit@am.iitd.ac.in & rajdipn@am.i

Webpage: **https://rajdip-iitd.github.io/bechtel2/**

Neural Networks

- We already looked at two basic parametric models → linear regression, logistic regression

- A neural network, in some sense, extends these basic models by stacking multiple copies of these models to construct a hierarchical model

- This hierarchical model can describe more complicated relationships between inputs and outputs than a linear or logistic regression

- Deep learning is a subfield of machine learning that deals with such hierarchical machine learning models

Neural Network Model

- Earlier we introduced the concept of non-linear parametric functions for modelling the relationship between input variables $x_1, \ldots, x_p$ and output y

$$\hat{y} = f_{\underline{\theta}}(\underline{x}) = f_{\underline{\theta}}(x_1, x_2, \ldots, x_p)$$

the function f is "parameterized" by $\underline{\theta}$

- Such a non-linear function $f_{\underline{\theta}}(\cdot)$ can be created in many ways

- In neural network, the strategy is to use several layers of linear regression models and non-linear activation functions

Neural Network model – linear regression

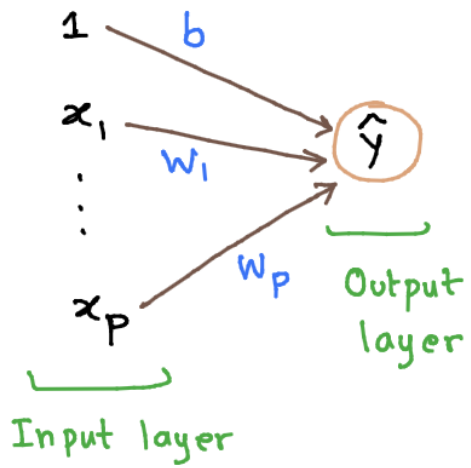
- We start the description of a neural network model with linear regression model

$$\hat{y} = \underbrace{b}_{\theta_0} + \underbrace{w_1 x_1}_{\theta_1} + \underbrace{w_2 x_2}_{\theta_2} + \cdots + \underbrace{w_p x_p}_{\theta_p}$$

$w_1, w_2, \ldots, w_p$ are called the weights and b is the offset

we use this notation because it is more popular in neural networks

- Graphical representation

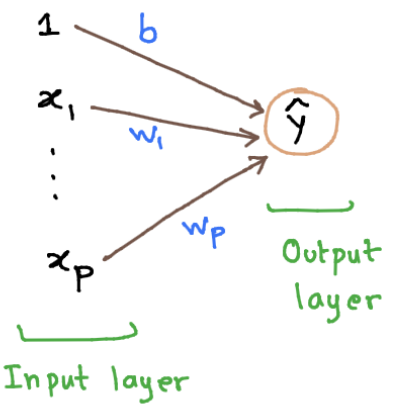


$$\hat{y} = w_1 x_1 + \cdots + w_p x_p + b$$

w_j → – Each link is associated with a parameter

$\hat{y}$ – Output $\hat{y}$ is described as sum of all terms

Neural network model – nonlinear relationship



$$\hat{y} = w_1 x_1 + \cdots + w_p x_p + b$$

Describes a linear relationship

- How to describe a non-linear relationship between

$$\underline{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_p \end{bmatrix} \quad \text{and} \quad \hat{y} \; ?$$

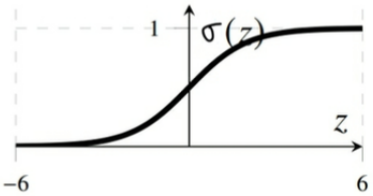
- Use activation function $h : \mathbb{R} \to \mathbb{R}$

$$\hat{y} = \sigma(w_1 x_1 + \cdots + w_p x_p + b)$$

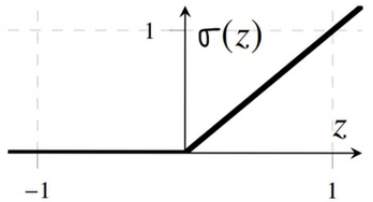
- Common choices of activation functions are:

Logistic (Sigmoid): $\sigma(z) = \dfrac{1}{1+e^{-z}}$

ReLU: $\sigma(z) = \max(0, z)$ (standard choice)



Logistic: $\sigma(z) = \frac{1}{1+e^{-z}}$



ReLU: $\sigma(z) = \max(0, z)$

Neural network – contd.

$$\hat{y} = \sigma\left(w_1 x_1 + \cdots + w_p x_p + b\right)$$

This now becomes a generalized linear model

- This generalized linear model is very simple
- Cannot describe very complicated relationships between input $\underline{x}$ and output $\hat{y}$

– How can we extend this simple model to increase the flexibility ?

Stack several generalized linear models in a sequential construction

Ex:

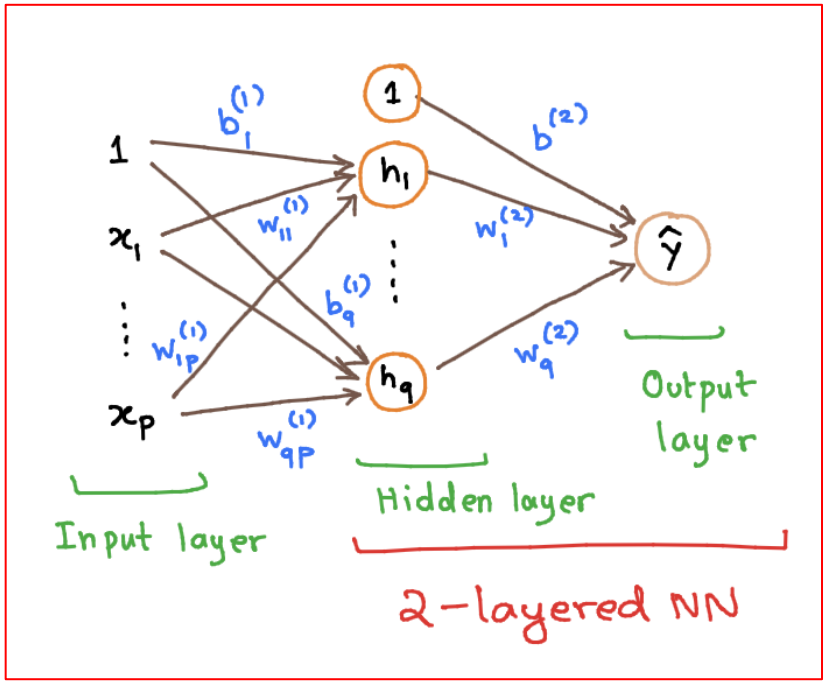
One-layer of stacking (hidden layer)

$$\begin{aligned}
h_1 &= \sigma\left(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + \cdots + w_{1p}^{(1)} x_p + b_1^{(1)}\right) \\
h_2 &= \sigma\left(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + \cdots + w_{2p}^{(1)} x_p + b_2^{(1)}\right) \\
&\vdots \\
h_q &= \sigma\left(w_{q1}^{(1)} x_1 + w_{q2}^{(1)} x_2 + \cdots + w_{qp}^{(1)} x_p + b_q^{(1)}\right)
\end{aligned}$$

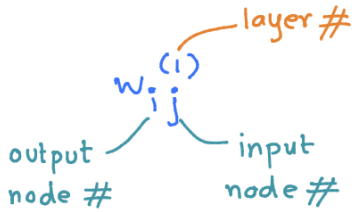
Output layer

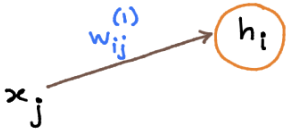
$$\hat{y} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + \cdots + w_q^{(2)} h_q + b^{(2)}$$

Two layered Neural Network

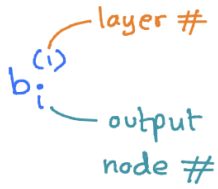


- Each link (arrow) is associated with a weight





- Each circular node is associated with its own bias term b



$$h_1 = \sigma\left(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + \cdots + w_{1p}^{(1)} x_p + b_1^{(1)}\right)$$

$$h_2 = \sigma\left(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + \cdots + w_{2p}^{(1)} x_p + b_2^{(1)}\right)$$

$$\vdots \qquad\qquad \vdots$$

$$h_q = \sigma\left(w_{q1}^{(1)} x_1 + w_{q2}^{(1)} x_2 + \cdots + w_{qp}^{(1)} x_p + b_q^{(1)}\right)$$

$$\hat{y} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + \cdots + w_q^{(2)} h_q + b^{(2)}$$

Two layered Neural Network – Vectorized representation

- The 2-layered neural network can be more compactly written using matrix notation:

$$\underline{\underline{W}}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & \cdots & w_{1p}^{(1)} \\ \vdots & & \vdots \\ w_{q1}^{(1)} & \cdots & w_{qp}^{(1)} \end{bmatrix}_{q \times p}, \quad \underline{b}^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_q^{(1)} \end{bmatrix}_{q \times 1}, \quad \underline{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix}_{p \times 1}$$

$$\underline{\underline{W}}^{(2)} = \begin{bmatrix} w_1^{(2)} & \cdots & w_q^{(2)} \end{bmatrix}_{1 \times q}, \quad \underline{b}^{(2)} = \begin{bmatrix} b^{(2)} \end{bmatrix}_{1 \times 1}, \quad \underline{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_q \end{bmatrix}_{q \times 1}$$

Compact Representation

$$\underset{q \times 1}{\underline{h}} = \sigma\left(\underbrace{\underline{\underline{W}}^{(1)} \underline{x} + \underline{b}^{(1)}}_{q \times 1} \right)$$

$$\underset{1 \times 1}{\hat{y}} = \underset{1 \times q}{\underline{\underline{W}}^{(2)}} \underset{q \times 1}{\underline{h}} + \underset{1 \times 1}{\underline{b}^{(2)}}$$

Parameters

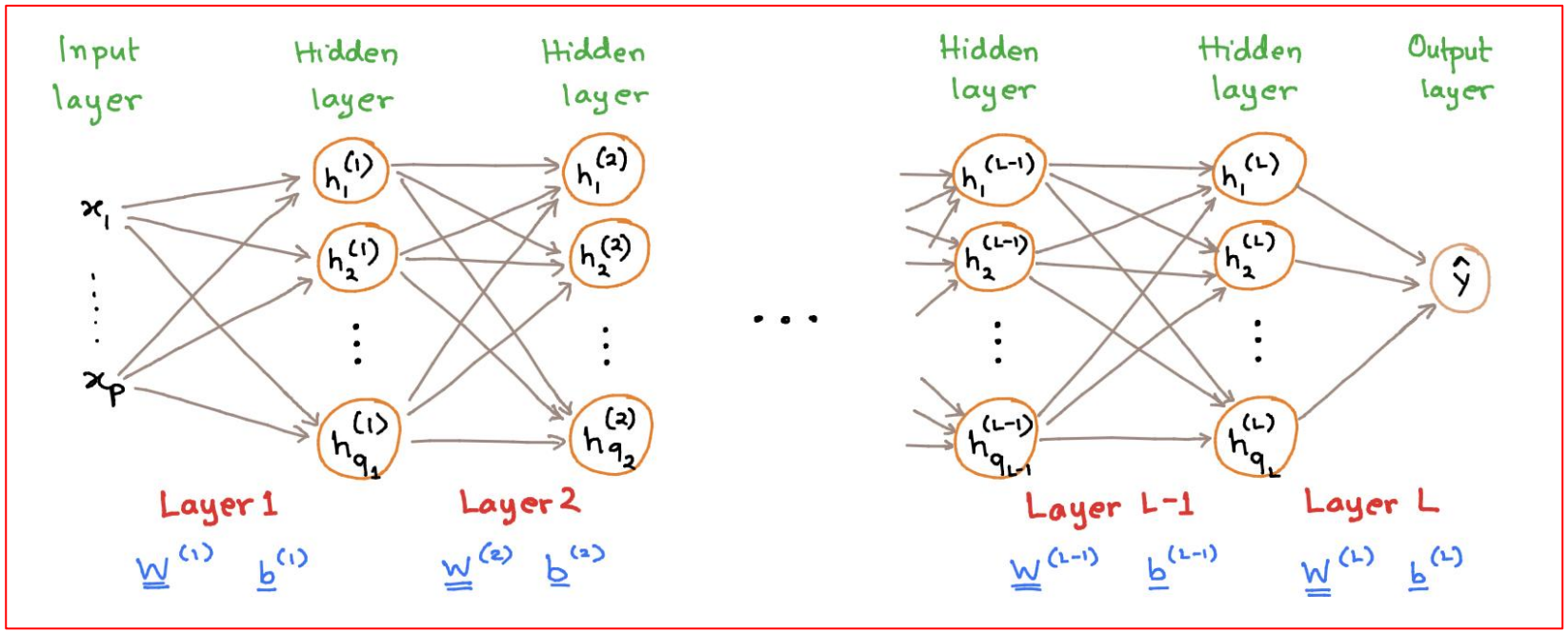
$$\underline{\theta} = \begin{bmatrix} \text{vec}(\underline{\underline{W}}^{(1)}) \\ \underline{b}_1^{(1)} \\ \text{vec}(\underline{\underline{W}}^{(2)}) \\ b^{(2)} \end{bmatrix}$$

Regression problem

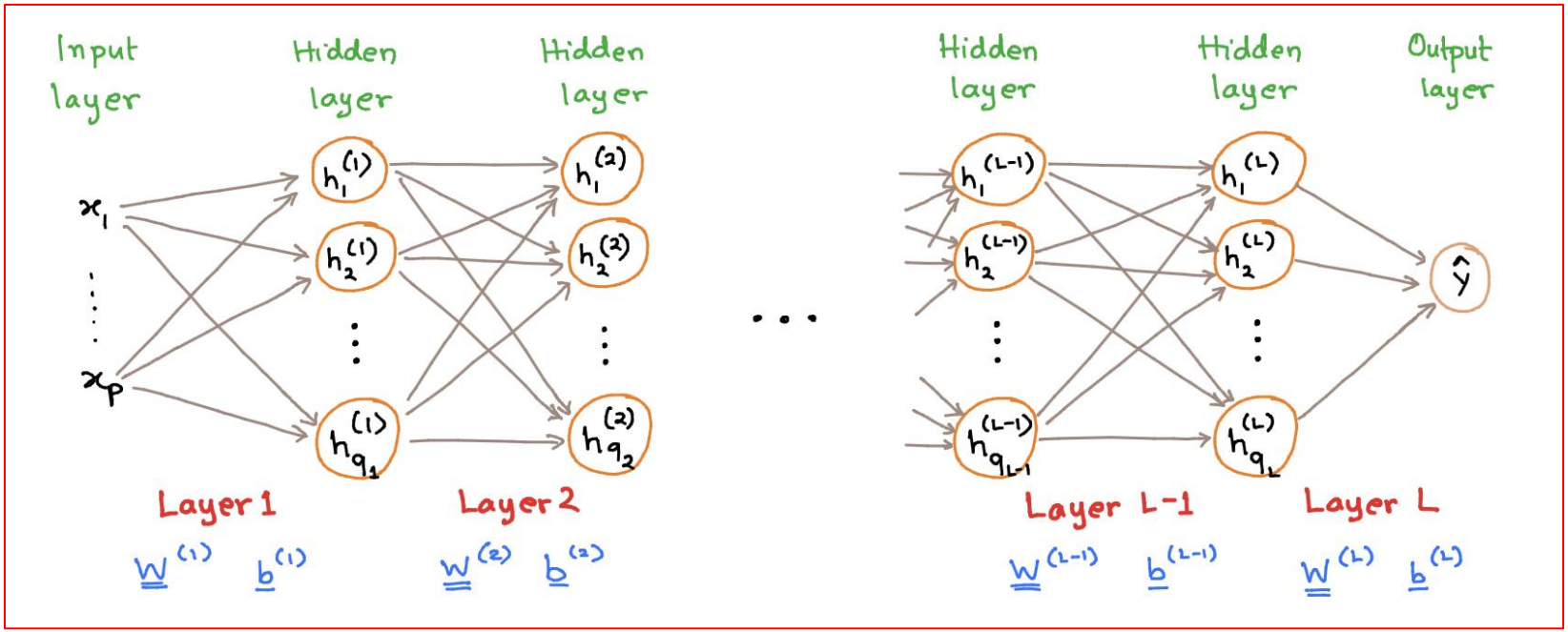
$$\hat{y} = f_{\underline{\theta}}(\underline{x})$$

Deep Neural Network

- The 2-layered neural network is called shallow NN (because it has one hidden layer)
- The real flexibility of neural network comes when we have more than one hidden layer.
- Stacking multiple hidden layers leads to a DEEP neural network



Deep Neural Network (Feed-Forward Neural Network - FNN)



Mathematical representation of FNN:

$$\underline{h}^{(1)} = \sigma\left(\underline{\underline{W}}^{(1)}\,\underline{x} + \underline{b}^{(1)}\right)$$

$$\underline{h}^{(2)} = \sigma\left(\underline{\underline{W}}^{(2)}\,\underline{h}^{(1)} + \underline{b}^{(2)}\right)$$

$$\vdots$$

$$\underline{h}^{(L)} = \sigma\left(\underline{\underline{W}}^{(L-1)}\,\underline{h}^{(L-1)} + \underline{b}^{(L-1)}\right)$$

$$\hat{y} = \underline{\underline{W}}^{(L)}\,\underline{h}^{(L-1)} + \underline{b}^{(L)}$$

Vectorization over data-points

- During training, the neural network model is used to compute the predicted output for several input points $\{ y_i , \underline{x}_i \}_{i=1}^{N}$

- The 2-layer neural network

$$\underset{q\times 1}{\underline{h}} = \sigma\left(\underset{q\times 1}{\underline{\underline{W}}^{(1)}\,\underline{x}} + \underline{b}^{(1)} \right)$$

$$\underset{1\times 1}{\hat{y}} = \underset{1\times q}{\underline{\underline{W}}^{(2)}}\,\underset{q\times 1}{\underline{h}} + \underset{1\times 1}{\underline{b}^{(2)}}$$

(column vector)

→ (row vector)

$$\underset{1\times q}{\underline{h}_i^T} = \sigma\left(\underset{1\times p}{\underline{x}_i^T}\,\underset{p\times q}{\underline{\underline{W}}^{(1)T}} + \underset{1\times q}{\underline{b}^{(1)T}} \right)$$

$$\underset{1\times 1}{\hat{y}_i} = \underset{1\times q}{\underline{h}_i^T}\,\underset{q\times 1}{\underline{\underline{W}}^{(2)T}} + \underset{1\times 1}{\underline{b}^{(2)T}}$$

$i = 1, 2, \ldots, n$

Transposed

Holds for each data point

Stack all data points in matrices

$$\underset{N\times 1}{\underline{y}} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}, \quad \underset{N\times p}{\underline{\underline{X}}} = \begin{bmatrix} \underline{x}_1^T \\ \vdots \\ \underline{x}_N^T \end{bmatrix}, \quad \underset{N\times 1}{\underline{\hat{y}}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_N \end{bmatrix}, \quad \underset{N\times q}{\underline{\underline{H}}} = \begin{bmatrix} \underline{h}_1^T \\ \vdots \\ \underline{h}_N^T \end{bmatrix}$$

each row represents a training data point

$$\underline{\underline{H}} = \sigma\left(\underline{\underline{X}}\,\underline{\underline{W}}^{(1)T} + \underline{b}^{(1)T} \right)$$

$$\underline{\hat{y}} = \underline{\underline{H}}\,\underline{\underline{W}}^{(2)T} + \underline{b}^{(2)T}$$

To be implemented in PyTorch, Tensorflow.

Neural Network for classification

— How did we extend linear regression to logistic regression?

- By applying logistic (or sigmoid) function to the output of linear regression in case of binary classification
- For multi-class classification (with $y \in \{1, 2, \ldots, M\}$ classes), we used the softmax function

$$\text{softmax}(\underline{z}) = \frac{1}{\sum_{j=1}^{M} e^{z_j}} \begin{bmatrix} e^{z_1} \\ e^{z_2} \\ \vdots \\ e^{z_M} \end{bmatrix}$$

— The softmax function now becomes an additional activation function, acting on the final layer of the neural network

$$\underline{h}^{(1)} = \sigma\left(\underline{\underline{W}}^{(1)} \underline{x} + \underline{b}^{(1)}\right)$$

$$\vdots$$

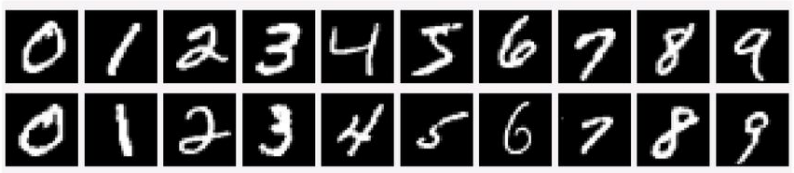
$$\underline{h}^{(L-1)} = \sigma\left(\underline{\underline{W}}^{(L-1)} \underline{h}^{(L-2)} + \underline{b}^{(L-1)}\right)$$

$$\underset{M\times 1}{\underline{z}} = \underline{\underline{W}}^{(L)} \underline{h}^{(L-1)} + \underline{b}^{(L)}$$

$$\underset{M\times 1}{\underline{g}} = \text{softmax}(\underline{z})$$

Classification example

- Dataset has 60000 training images
- " " 10000 test/validation images
- Each data point consists of a 28×28 pixel grayscale image of a handwritten digit
- Each image is also labelled with the digit $0, 1, 2, \ldots, 9$ that it depicts
- Each pixel intensity has been normalized between $[0, 1]$



- Consider image as input $\underline{x} = [x_1 \; x_2 \; \ldots \; x_p]^T$
  - $p = 28 \times 28 = 784$ input variables (flattened out)
  - Each x_j corresponds to a pixel in the image and represents its intensity

$x_j = 0 \longrightarrow$ black pixel

$x_j = 1 \longrightarrow$ white pixel

Using a 2-layer NN

Consider 200 hidden units

$$\underset{60000\times200}{\underline{\underline{H}}} = \sigma\left(\underset{60000\times784}{\underline{\underline{X}}} \; \underset{784\times200}{\underline{\underline{W}}^{(1)^T}} + \underset{1\times200}{\underline{b}^{(1)^T}} \right)$$

$$\underset{60000\times10}{\hat{\underline{y}}} = \underset{}{\underline{\underline{H}}} \; \underset{200\times10}{\underline{\underline{W}}^{(2)^T}} + \underset{1\times10}{\underline{b}^{(2)^T}}$$

Backpropagation for training Neural Networks

- A neural network is a parametric model
- We will tune its parameters using optimization techniques such as SGD, ADAM, etc.
- To find suitable values for the parameters $\underline{\theta}$, we will solve the optimization problem:

$$\hat{\underline{\theta}} = \underset{\underline{\theta}}{\operatorname{argmin}} \underbrace{J(\underline{\theta})}_{\text{cost function}}$$

where $J(\underline{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \underbrace{L\left(y_i, f(\underline{x}_i; \underline{\theta})\right)}_{\text{loss function for data point } (\underline{x}_i, y_i)}$

$$\underline{\theta} = \begin{bmatrix} \text{vec}(\underline{\underline{W}}^{(1)}) \\ \underline{b}^{(1)} \\ \vdots \\ \text{vec}(\underline{\underline{W}}^{(L)}) \\ \underline{b}^{(L)} \end{bmatrix}$$

Training Neural Network

- The functional form of the loss function depends on the problem
  - Regression problems typically use squared error loss

$$L\left(y, f(\underline{x}; \underline{\theta})\right) = \left(y - f(\underline{x}; \underline{\theta})\right)^2$$

$f(\underline{x}; \underline{\theta})$: output of a neural net

- Multi-class classification problems typically use cross-entropy loss (with M classes)

$$L\left(y, f(\underline{x}; \underline{\theta})\right) = -\ln g_m\left(\underline{f}(\underline{x}; \underline{\theta})\right)$$

g_m: softmax

$\underline{f}(\underline{x}; \underline{\theta})$: is a vector of $M \times 1$

$$= -\ln g_y\left(\underline{f}(\underline{x}; \underline{\theta})\right)$$

g_y: we use training data label y as an index variable to select the correct logit

- These optimization problems cannot be solved in closed form
  - → Numerical optimization algorithms have to be used

Training Neural Networks

- Numerical optimization update parameters in an iterative manner

  In deep learning, one typically uses gradient-descent algorithms

Step 1 : Pick an initial guess $\underline{\theta}^{(0)}$

Step 2 : Calculate gradient of the cost function w.r.t $\underline{\theta}^{(t)}$, $t = 0, 1, 2, \ldots$

Step 3 : Update the parameters as $\underline{\theta}^{(t+1)} \leftarrow \underline{\theta}^{(t)} - \gamma \nabla_{\underline{\theta}} J(\underline{\theta}^{(t)})$

Step 4 : Terminate when some criterion is fulfilled, and take the last $\underline{\theta}^{(t)}$ as $\hat{\underline{\theta}}$

Training Neural Networks

Computational Challenges

1) Large datasets (N very large)

- The number of datapoints N is large in deep learning applications
- Makes computation of the cost function gradient very costly, due to the sum
- We resort to using a random subset of data to update parameters
  → minibatch gradient descent

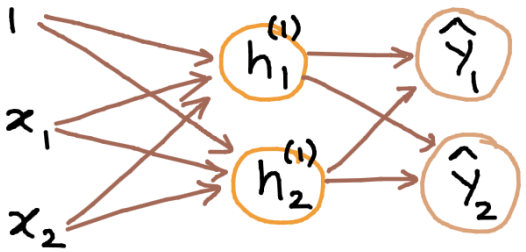
2) Large number of parameters $\underline{\theta}$

- The dimension of the parameter vector $\underline{\theta}$ is very large in deep learning
- To efficiently calculate the gradient $\nabla_{\underline{\theta}} J(\underline{\theta}^{(t)})$, we need to apply chain rule of calculus

this will be done using the Backpropagation algorithm

Backpropagation

Neural net with 1-hidden layer (with multiple outputs)



Forward pass (to compute loss)

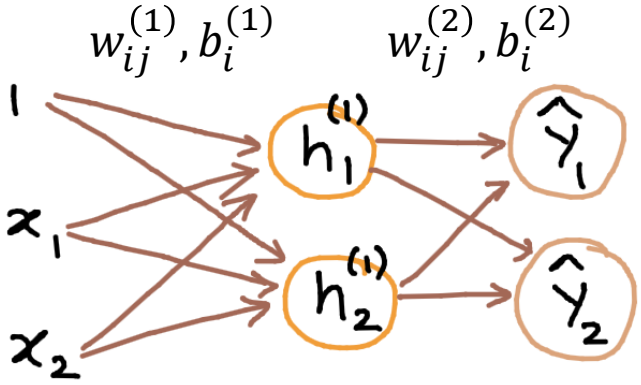
$$z_i^{(1)} = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i^{(1)} = \sigma\left(z_i^{(1)}\right)$$

$$\hat{y}_K = \sum_i w_{Ki}^{(2)} h_i^{(1)} + b_K^{(2)}$$

$$L = \sum_K \left(y_K - \hat{y}_K\right)^2$$

Backpropagation – contd.



Forward pass
(to compute loss)

$$z_i^{(1)} = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i^{(1)} = \sigma\left(z_i^{(1)}\right)$$

$$\hat{y}_k = \sum_i w_{ki}^{(2)} h_i^{(1)} + b_k^{(2)}$$

$$L = \sum_k \left(y_k - \hat{y}_k\right)^2$$

Backward pass
(to compute gradients)

$$\frac{\partial L}{\partial \hat{y}_k} = -2(y_k - \hat{y}_k)$$

1. $$\frac{\partial L}{\partial w_{ki}^{(2)}} = \frac{\partial L}{\partial \hat{y}_k}\frac{\partial \hat{y}_k}{\partial w_{ki}^{(2)}} = -2(y_k - \hat{y}_k)h_i^{(1)}$$

2. $$\frac{\partial L}{\partial b_k^{(2)}} = \frac{\partial L}{\partial \hat{y}_k}\frac{\partial \hat{y}_k}{\partial b_k^{(2)}} = -2(y_k - \hat{y}_k)$$

$$\frac{\partial L}{\partial h_i^{(1)}} = \sum_k \frac{\partial L}{\partial \hat{y}_k}\frac{\partial \hat{y}_k}{\partial h_i^{(1)}} = \sum_k \left[-2(y_k - \hat{y}_k)w_{ki}^{(2)}\right]$$

$$\frac{\partial h_i^{(1)}}{\partial z_i^{(1)}} = \sigma'(z_i^{(1)})$$

3. $$\frac{\partial L}{\partial w_{ij}^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}}\frac{\partial h_i^{(1)}}{\partial z_i^{(1)}}\frac{\partial z_i^{(1)}}{\partial w_{ij}^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}}\sigma'(z_i^{(1)})x_j$$

4. $$\frac{\partial L}{\partial b_i^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}}\frac{\partial h_i^{(1)}}{\partial z_i^{(1)}}\frac{\partial z_i^{(1)}}{\partial b_i^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}}\sigma'(z_i^{(1)})$$

Introduction to basic parametric models

- Backprop is based on the computational graph, and it basically works backwards through the graph, applying the chain rule at each node
- Backprop is used to train most neural nets you will find these days
- Even optimization algorithms much fancier than gradient descent (such as second-order methods) use backprop to compute gradients
- Once the derivatives w.r.t. the weights and biases are computed using backprop, the updates are applied to the weights and biases using some optimization scheme

$$\underline{\underline{W}}^{(t+1)} \leftarrow \underline{\underline{W}}^{(t)} - \eta \left.\frac{\partial J}{\partial \underline{\underline{W}}}\right|_{\underline{\underline{W}}^{(t)}} \qquad \underline{b}^{(t+1)} \leftarrow \underline{b}^{(t)} - \eta \left.\frac{\partial J}{\partial \underline{b}}\right|_{\underline{b}^{(t)}}$$

- Hand-calculation of derivatives are replaced with automatic differentiation